30-Day Android Challenge

Senior Android interview prep — Junior → Mid → Senior → Architect

2026 Edition

DAY 25

Modularization & Dynamic Features — api/impl, Dependency Graphs & Play Feature Delivery



Esmaeeil Moradi
@esmaeeil-moradi700

DAY 25

Why modularization matters in 2026

A monolith compiles, tests and gets reviewed as one unit.
Modules turn architecture rules into **compiler rules**:

- **Build speed:** a change rebuilds its module and its dependents; independent modules build in parallel and come from the build cache
- **Enforced boundaries:** `internal` and `implementation` make "don't touch that" a compile error
- **Ownership:** a team owns a module, its API and its reviews
- **Reuse:** the same modules in the phone app, a demo app, Wear OS or KMP
- **Delivery:** Play Feature Delivery can install parts of the app later — at a real cost

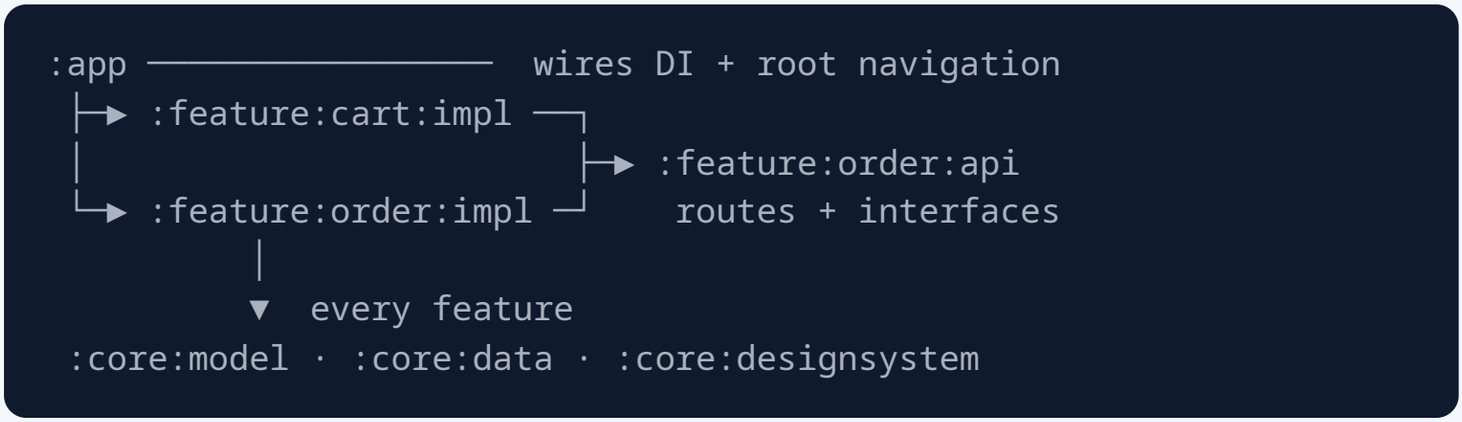
Level	Questions
Junior	Why modularize · Module types · Dependency direction
Mid	api/impl split · Navigation between features · DI across modules · Resources & core modules
Senior	Build speed: what really gets faster · Play Feature Delivery · Its costs & alternatives
Architect	Governing the graph · Migrating a monolith · Granularity & demo apps



Esmaeeil Moradi
@esmaeeil-moradi700

DAY 25

Mental model: features talk through contracts



- **`:app` is thin:** `Application`, DI graph, root navigation — no feature logic
- **Features talk through `:api`:** routes and interfaces, never another feature's `:impl`
- **Core modules are shared and stable:** they never depend on features

THE RULE BEHIND EVERY MODULAR CODEBASE

Depend on what rarely changes (contracts, core), never on what changes daily (another feature's internals). Then a change stays inside one module.



Esmaeeil Moradi
@esmaeeil-moradi700

DAY 25

Junior

Why modularize · Module types · Dependency direction



Esmaeeil Moradi
@esmaeeil-moradi700

Q1. Why modularize — and what it costs

You gain	You pay
Incremental builds recompile only affected modules	Configuration and task overhead per module
Independent modules build in parallel and hit the build cache	More build files to keep consistent (convention plugins, Day 21)
Real encapsulation: `internal`, `implementation`	Contracts between modules must be designed
Clear ownership, fewer merge conflicts	Cross-module refactors take more steps
One feature can be built, tested and run alone	Navigation and DI need explicit wiring

INTERVIEW TRAP

"Modularization always makes builds faster." — It speeds up **incremental** builds and cached CI builds. A clean build of many small Android modules can even get slower: each module brings its own configuration, resource, manifest and lint tasks. Measure before and after.

FOLLOW-UP

"Why is `internal` a reason to modularize?" — In Kotlin, `internal` means *visible inside this Gradle module*. In a monolith every class is reachable from everywhere; in modules the compiler hides implementation classes.



Esmaeeil Moradi
@esmaeeil-moradi700

Q2. Module types

Type	Contains	Depends on
`:app`	`Application`, root navigation, DI wiring	Features, core
Feature	Screens + ViewModels of one user-facing feature	Core, other features' `:api`
Data	Repositories and data sources of one domain area	Core (model, network, database)
Core	Design system, models, networking, utilities	Other core modules only
Test	Fakes, test rules, fixtures (`:core:testing`)	The modules they fake
Dynamic feature	Code Play can deliver later	The base `:app` — **inverted**

- Code that needs no Android API goes into **plain Kotlin/JVM or KMP modules**: fewer tasks, faster builds, ready for sharing

INTERVIEW TRAP

"Put shared code in one `:core` module that every module depends on." (last year's advice) — It becomes a junk drawer: every change invalidates the whole graph above it. Split by purpose: `:core:model`, `:core:network`, `:core:designsystem`.



Esmaeeil Moradi
@esmaeeil-moradi700

Q3. Dependency direction

- Dependencies point **one way**: app → feature → data → core. Core never knows features.
- A feature may depend on another feature's **`:api`** — never on its `:impl`.
- Only `:app` (or a feature's demo app) depends on `:impl` modules.

```
// feature/cart/impl/build.gradle.kts
plugins {
    id("app.android.feature") // convention plugin (Day 21)
}

dependencies {
    implementation(project(":feature:order:api"))
    implementation(project(":core:data"))
    implementation(project(":core:designsystem"))
    // implementation(project(":feature:order:impl")) — never
}
```

INTERVIEW TRAP

"The structure app → feature → data → core prevents circular dependencies." — Gradle already fails the build on a project cycle. The real damage comes from **legal but bad** edges: feature → another feature's `:impl`, or everything → one god `:core`. Check the graph in CI (Q11).



Esmaeeil Moradi
@esmaeeil-moradi700

DAY 25

Mid-Level

api/impl split · Navigation between features · DI across modules · Resources & core modules



Esmaeeil Moradi
@esmaeeil-moradi700

Q4. The api/impl split

```
:feature:order:api       small · stable · rarely changes
  OrderRoute, interface OrderRepository
        ▲ implements                ▲ uses
:feature:order:impl          :feature:cart:impl
  screens, ViewModels,
  DefaultOrderRepository, DI module
```

`:api`	`:impl`
Routes, public interfaces, the models they use	Screens, ViewModels, repositories, DI bindings
Few dependencies — no Room, no Compose UI	Whatever it needs
Changes rarely, so consumers rarely recompile	Changes daily — rebuilds itself, re-packages `:app`, other features untouched



Esmaeeil Moradi
@esmaeeil-moradi700

Q4. What it looks like

```
// :feature:order:api
@Serializable data class OrderRoute(val orderId: String)

interface OrderRepository {
    fun observeOpenCount(): Flow<Int>
}

// :feature:order:impl
internal class DefaultOrderRepository @Inject constructor(
    private val dao: OrderDao,
) : OrderRepository {
    override fun observeOpenCount(): Flow<Int> =
        dao.observeOpenCount()
}
```

INTERVIEW TRAP

"An `:api` module means using Gradle's `api()` configuration." — Unrelated. `:api` is a module naming convention. `api()` vs `implementation()` decides whether a dependency leaks to consumers' compile classpath (Day 22). Inside both modules, prefer `implementation`.

INTERVIEW TRAP

"Every module needs an api/impl pair." — Only modules that other features consume. Splitting everything doubles the module count and its overhead.



Esmaeeil Moradi
@esmaeeil-moradi700

Q5. Navigation between feature modules

```
// :feature:cart:impl — knows nothing about orders
fun NavGraphBuilder.cartScreen(onCheckedOut: (String) -> Unit) {
    composable<CartRoute> {
        CartScreen(onCheckedOut = onCheckedOut)
    }
}

// :app — the only place that connects features
NavHost(navController, startDestination = CartRoute) {
    cartScreen(onCheckedOut = { orderId ->
        navController.navigate(OrderRoute(orderId))
    })
    orderScreen(onBack = { navController.popBackStack() })
}
```

- Each feature exposes a **route** and a **`NavGraphBuilder` extension**; `:app` connects features with lambdas — so features don't depend on each other at all
- If a feature must open another directly, it depends on that feature's `:api` route — never its screens



Esmaeeil Moradi
@esmaeeil-moradi700

Q5. Scaling it

- Many teams: each `:impl` **contributes its graph** through DI multibinding and `:app` assembles the root graph (Day 2, Q11)
- Navigation 3 keeps the same split: features provide entries, the app owns the back stack
- Route arguments live in the back-stack entry's saved state, so they survive process death on both sides of the module boundary

INTERVIEW TRAP

"Use deep-link URIs as the contract between modules." — Strings aren't compile-checked: a renamed path or argument breaks only at runtime. Use type-safe routes from `:api` inside the app; keep URIs for real external deep links.

FOLLOW-UP

"Where do cross-feature flows like checkout live?" — In a nested graph owned by one feature, entered through a single route in its `:api`. Other features only know that entry route.



Esmaeeil Moradi
@esmaeeil-moradi700

Q6. DI across modules

```
// :feature:order:impl — the binding lives with the impl
@Module
@InstallIn(SingletonComponent::class)
abstract class OrderDataModule {
    @Binds
    internal abstract fun bindOrderRepository(
        impl: DefaultOrderRepository,
    ): OrderRepository
}

// :feature:cart:impl — sees only :feature:order:api
@HiltViewModel
class CartViewModel @Inject constructor(
    private val orders: OrderRepository,
) : ViewModel()
```

- Hilt generates the components in `:app`, so every Hilt module must be in `:app`'s transitive dependencies; the Hilt Gradle plugin's aggregating task makes plain `implementation` edges enough
- Koin: each module exposes a `module { … }`; `:app` lists them in `startKoin { modules(…) }`



Esmaeeil Moradi
@esmaeeil-moradi700

Q6. Dynamic features and hidden coupling

INTERVIEW TRAP

"Hilt works the same in a dynamic feature module." — It doesn't: the feature depends on the app, not the other way round, so Hilt can't generate its components. Use plain Dagger in the feature and pull app dependencies through an `@EntryPoint`.

```
// :app
@EntryPoint
@InstallIn(SingletonComponent::class)
interface VideoCallDependencies {
    fun sessionRepository(): SessionRepository
}

// :videocall (dynamic feature)
val deps = EntryPointAccessors.fromApplication(
    applicationContext, VideoCallDependencies::class.java,
)
DaggerVideoCallComponent.builder()
    .videoCallDependencies(deps).build().inject(this)
```

INTERVIEW TRAP

"A global event bus fully decouples modules." (last year's advice) — It hides the coupling instead: both sides still share event types and timing, but nothing shows who depends on whom. Prefer an interface in `:api` or state exposed as a `Flow`.



Esmaeeil Moradi
@esmaeeil-moradi700

Q7. Resources and core UI modules

- Each module has its own `namespace` and — by default since AGP 8.0 — a **non-transitive R class**: `R` holds only that module's resources
- `:core:designsystem`: theme, components, icons — no domain models. `:core:ui`: shared composables that do know domain models
- Feature strings stay in the feature: a translation update rebuilds one module
- Same resource name in two modules: the app wins over libraries, silently. `resourcePrefix` makes Lint flag unprefixed names

```
// core/designsystem/build.gradle.kts
android {
    namespace = "com.app.core.designsystem"
    resourcePrefix = "ds_"
}

// any feature
import com.app.core.designsystem.R as DsR
Text(stringResource(DsR.string.ds_retry))
```

INTERVIEW TRAP

"Share values through Gradle properties and `resValue`." (last year's advice) — That's build-time config, not a resource strategy: no translations, no tooling. And an "API key" injected this way is readable in the APK.



Esmaeeil Moradi
@esmaeeil-moradi700

DAY 25

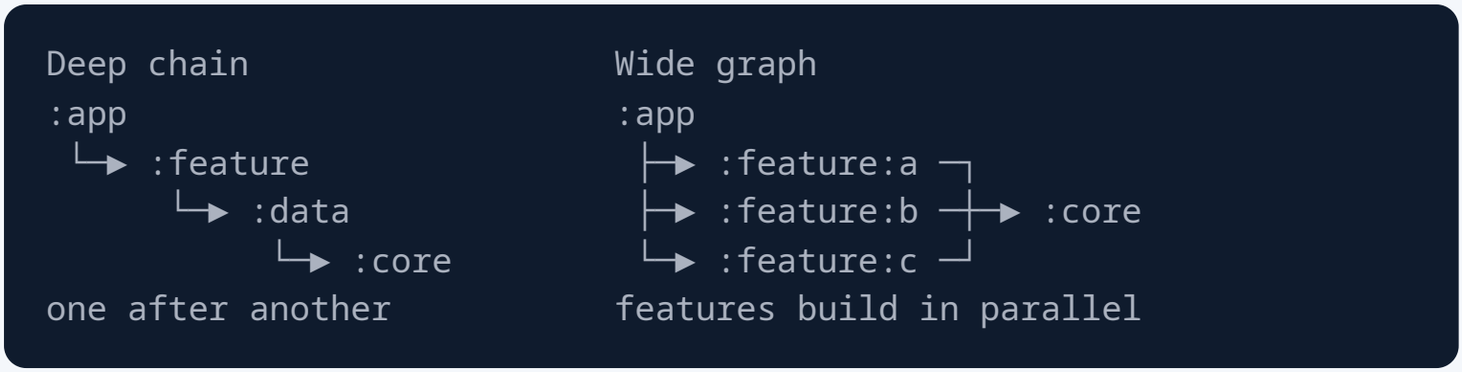
Senior

Build speed: what really gets faster · Play Feature Delivery · Its costs & alternatives



Esmaeeil Moradi
@esmaeeil-moradi700

Q8. Build speed: what really gets faster



Mechanism	Needs
Compile avoidance: dependents skip recompiling when a module's ABI doesn't change	Changes behind a stable public API; `implementation` edges
Parallel execution of independent modules	A wide, shallow graph; `org.gradle.parallel=true`
Build cache: unchanged modules come from a local or remote cache	Cacheable tasks; a remote cache fed by CI
Configuration cache: repeat builds skip configuration	Compatible plugins (Day 21)



Esmaeeil Moradi
@esmaeeil-moradi700

Q8. Where the time goes

- The **longest path** in the graph is the floor: a deep chain builds in sequence no matter how many cores you have
- A public signature change in a low-level module ripples through everything above it — keep core APIs **small and stable**
- Android library modules carry many more tasks than plain Kotlin/JVM modules
- KSP instead of kapt in every module (Day 21)
- Measure: Build Analyzer, Build Scans, `gradle-profiler` for repeatable benchmarks

INTERVIEW TRAP

"Change one file and only its module recompiles." (last year's claim) — Only while its ABI stays the same. Change a public signature in `:core:model` and every dependent recompiles. And `:app` still re-dexes and re-packages on every change.

FOLLOW-UP

"Clean builds got slower after modularizing. Did it fail?" — Not necessarily. Judge by incremental builds for typical changes and by CI time with a warm cache — that's what developers actually wait for.



Esmaeeil Moradi
@esmaeeil-moradi700

Q9. Play Feature Delivery: the options

Delivery	Installed	Use for
install-time	With the app	Modular code that is always needed
conditional	At install, if the device matches: device feature, min SDK, country	AR, hardware-specific features
on-demand	When the app requests it	Large, optional, rarely used features

For large **assets** (ML models, game data) use **Play Asset Delivery** instead: install-time, fast-follow and on-demand asset packs — no code split needed.

```
// app/build.gradle.kts — the base module
android {
    dynamicFeatures += setOf(":videocall")
}

// videocall/build.gradle.kts
plugins { id("com.android.dynamic-feature") }
dependencies { implementation(project(":app")) }
```



Esmaeeil Moradi
@esmaeeil-moradi700

Q9. Declaring delivery

```
<!-- videocall/src/main/AndroidManifest.xml -->
<manifest
    xmlns:dist="http://schemas.android.com/apk/distribution">
    <dist:module dist:title="@string/title_videocall">
        <dist:delivery>
            <dist:on-demand />
        </dist:delivery>
        <dist:fusing dist:include="true" />
    </dist:module>
</manifest>
```

- `dist:title` is shown to the user, so on-demand modules need one
- `dist:fusing` decides whether the module is included in a universal APK

INTERVIEW TRAP

"Configure conditional delivery in `build.gradle.kts`." (last year's answer) — Conditions are declared in the module's **manifest**, inside `<dist:install-time>`: `<dist:conditions>` with `<dist:device-feature>`, `<dist:min-sdk>` or `<dist:user-countries>`.



Esmaeeil Moradi
@esmaeeil-moradi700

Q9. Installing on demand

```
// import com.google…SplitInstallSessionStatus as Status
val splits = SplitInstallManagerFactory.create(context)

val listener = SplitInstallStateUpdatedListener { state ->
    when (state.status()) {
        Status.DOWNLOADING -> showProgress(state)
        Status.REQUIRES_USER_CONFIRMATION -> splits
            .startConfirmationDialogForResult(state, launcher)
        Status.INSTALLED -> openVideoCall()
        Status.FAILED -> showError(state.errorCode())
    }
}
splits.registerListener(listener) // unregister when done
val request = SplitInstallRequest.newBuilder()
    .addModule("videocall")
    .build()
splits.startInstall(request)
```

- Call `SplitCompat.installActivity(this)` in the module's activities (and `SplitCompat.install` in the base `Application`) to use new code without a restart

INTERVIEW TRAP

"Download it with the Play Core library." — The monolithic Play Core library is deprecated and not compatible with targetSdk 34+. Use `com.google.android.play:feature-delivery` (plus the separate Asset Delivery, In-App Updates and In-App Review libraries).



Esmaeeil Moradi
@esmaeeil-moradi700

Q10. The real costs of dynamic features

Cost	Why
Inverted dependency	The base can't reference module classes: load entry points by name (reflection, `ServiceLoader`) and keep them in R8 rules
DI	Hilt can't process dynamic features — Dagger + `@EntryPoint` (Q6)
Testing	A normal debug run installs everything; test real delivery with `bundletool --local-testing` or internal app sharing
Distribution	Only Google Play delivers modules on demand; other stores and sideloads need a universal APK
UX	Progress, confirmation, failures, offline, low storage — for every entry point

Try these first: R8 and resource shrinking plus App Bundle splits (Day 18); Play Asset Delivery or your own server for big assets; plain library modules for code organization.

INTERVIEW TRAP

"Prefetch the Settings module because most users open it." — If most users need it, it shouldn't be on-demand at all. On-demand is for features a minority uses; `deferredInstall()` is only a best-effort background install.



Esmaeeil Moradi
@esmaeeil-moradi700

DAY 25

Architect

Governing the graph · Migrating a monolith · Granularity & demo apps



Esmaeeil Moradi
@esmaeeil-moradi700

Q11. Governing the module graph

Rule	Why
No `:impl` depends on another `:impl`	Features stay independently buildable
`:core:*` never depends on `:feature:*`	A stable base; no hidden cycles through helpers
`:api` modules stay light (no Room, no Compose UI)	Consumers stay cheap to compile
A maximum graph height	Short critical path, more parallelism

```
// app/build.gradle.kts — fails on a forbidden edge
plugins { id("com.jraska.module.graph.assertion") }

moduleGraphAssert {
    maxHeight = 5
    restricted = arrayOf(
        ":feature:.*:impl -X> :feature:.*:impl",
        ":core:.* -X> :feature:.*",
    )
}
```

Run `./gradlew assertModuleGraph` in CI so the rule fails a pull request, not a review.



Esmaeeil Moradi
@esmaeeil-moradi700

Q11. The governance toolbox

Tool	Enforces
modules-graph-assert	Allowed and forbidden edges, graph height
Dependency Analysis Gradle Plugin	Unused and misdeclared dependencies
Convention plugins (Day 21)	Every module is configured the same way
`CODEOWNERS` per module	The owning team reviews its module
Konsist (Day 3)	Code rules inside modules

INTERVIEW TRAP

"Check module rules with Dependency-Check." (last year's advice) — OWASP Dependency-Check scans libraries for known vulnerabilities (CVEs). It knows nothing about your module graph.

FOLLOW-UP

"Should a feature never depend on another feature?" — Never on its `:impl`. Depending on its `:api` is fine — that's what the contract is for.



Esmaeeil Moradi
@esmaeeil-moradi700

Q12. Migrating a monolith

1. **Measure first:** build times for typical changes; draw the current package dependency graph.
2. **Make modules cheap:** version catalog + convention plugins, so a new module takes minutes (Day 21).
3. **Untangle packages inside `:app`:** package by feature, remove cycles, enforce with Konsist. Moving clean packages into modules is the easy part.
4. **Extract leaves first:** `:core:model`, `:core:designsystem`, `:core:network` — code that depends on nothing app-specific.
5. **Then data, then features, one at a time:** start with the newest or least entangled.
6. **`:app` shrinks to a shell;** leftovers can live in a temporary `:legacy` module.

INTERVIEW TRAP

"Modularize on a long-running branch, then merge." — Months of merge conflicts while feature work continues. Do it on trunk: one extraction per pull request, each shippable.



Esmaeeil Moradi
@esmaeeil-moradi700

Q12. Breaking the cycles

```
// Before: profile opens OrderHistoryScreen and orders
// reads ProfileRepository — a cycle between two modules.
// After: profile navigates through :app (Q5), and orders
// sees only an interface — :feature:profile:api
interface ProfileReader {
    fun observeDisplayName(): Flow<String>
}

// :feature:order:impl depends on :feature:profile:api only
class OrderHeaderViewModel @Inject constructor(
    profile: ProfileReader,
) : ViewModel()
```

- **Dependency inversion:** the lower side declares the interface, the other side implements it, DI binds it
- Or move the shared part into a new, lower module both can depend on

INTERVIEW TRAP

"Start by moving all `Utils` *into* `:core:common`*."* — You've just moved the junk drawer, and now every module depends on it. Extract by purpose, or leave the helper where it's used.



Esmaeeil Moradi
@esmaeeil-moradi700

Q13. Granularity and demo apps

Too coarse	Too fine
One module for many features — most changes rebuild it	A module per screen or class — per-module overhead dominates
Several teams collide in the same module	Hard to navigate; contracts everywhere

- **One feature module = one capability owned by one team** — the same boundary as a domain context
- Split when several teams change one module; merge when two modules always change together

```
// feature/cart/demo/build.gradle.kts
plugins { id("com.android.application") }

dependencies {
    implementation(project(":feature:cart:impl"))
    implementation(project(":core:testing")) // fakes
}
```

A **demo app per feature** builds only what that feature needs and opens straight into it with fake data.



Esmaeeil Moradi
@esmaeeil-moradi700

Q13. What the interviewer wants to hear

INTERVIEW TRAP

"We have 800 modules, so our architecture is great." — Module count proves nothing. The questions are: what's the longest path, who owns each module, and how many modules does a typical feature change touch?

FOLLOW-UP

"How do you know modularization paid off?" — Track incremental build times for typical changes, CI duration, how many modules a typical pull request touches, and review time per team — before and after.

SENIOR ANSWER IN ONE LINE

"Modularize along ownership and change: what changes together lives together, what must not know each other is separated by an `:api` — and the graph rules are checked by CI, not by memory."



Esmaeeil Moradi
@esmaeeil-moradi700

DAY 25 · ARCHITECT

Day 25 — cheat sheet

- Modules turn architecture rules into compiler rules: `internal`, `implementation`, allowed edges.
- The wins are incremental builds, parallelism and caching — clean builds may not get faster. Measure.
- Types: app, feature, data, core, test, dynamic feature. Split `:core` by purpose — no junk drawer.
- Dependencies point one way; no feature depends on another feature's `:impl`.
- `:api` = routes + interfaces, small and stable. It is **not** Gradle's `api()` configuration.
- Navigation: features expose routes and graph builders; `:app` connects them with lambdas.
- DI: bindings live in `:impl`, Hilt aggregates in `:app`. Dynamic features: Dagger + `@EntryPoint`.
- Non-transitive R: each module owns its resources; `resourcePrefix` prevents silent clashes.
- Play Feature Delivery: install-time, conditional, on-demand — conditions in the manifest, Play Feature Delivery library, not Play Core.
- Govern the graph in CI; migrate incrementally on trunk, leaves first.

Day 26 is next → Advanced Kotlin. Save this post and follow along.



Esmaeeil Moradi
@esmaeeil-moradi700